

S2-MIÉ — RAG mínimo en ~60 líneas

MMIA 6013 · Semana 2 · Miércoles

Pipeline completo: chunking → índice → retrieval → prompt con contexto → LLM. Sin API funciona en "modo inspección": muestra el prompt armado y una respuesta pre-capturada, para que puedas depurar el retrieval igual.

Tarea real de MMIA 6013 — actividad de clase de la Semana 2, convertida de notebook a PDF para el solver (apartado 2.a del Taller 03 v2). Se conservan el texto y el código que da el profesor; las soluciones de los ejercicios se retiraron.

Entrega: un notebook de Jupyter (.ipynb) **ejecutado** de principio a fin sin errores, con una celda de Markdown que encabece cada parte, los ejercicios resueltos y las reflexiones respondidas con las cifras medidas.

Código dado:

```
import os
import json
import numpy as np
```

Código dado:

```
# Corpus: documentación interna de una empresa ficticia
DOCS = {
    "politica_vacaciones.md": """Política de vacaciones de NimbusSoft.
Los empleados a tiempo completo acumulan 1.5 días de vacaciones por mes trabajado,
hasta un máximo de 18 días por año. Las vacaciones deben solicitarse con al menos
15 días de anticipación a través del portal interno. Los días no utilizados pueden
transferirse al año siguiente hasta un máximo de 5 días. Durante el primer año,
los días solo pueden tomarse después de superar el período de prueba de 3
meses.""",
    "politica_remoto.md": """Política de trabajo remoto de NimbusSoft.
El trabajo remoto está permitido hasta 3 días por semana para todos los roles
excepto soporte de infraestructura on-site. Los días remotos se coordinan con el
líder de equipo. Para trabajar desde el exterior del país se requiere aprobación
de Recursos Humanos con 30 días de anticipación y un máximo de 60 días por año.""",
    "gastos.md": """Política de reembolso de gastos de NimbusSoft.
Los gastos de viaje se reembolsan presentando factura dentro de los 30 días
posteriores al gasto. El límite diario de alimentación en viajes es de 45 USD.
Los pasajes aéreos deben comprarse en clase económica salvo vuelos de más de
8 horas, donde se permite económica premium con aprobación del gerente de área.""",
}
```

Parte 1 — Chunking

🔗 Ejercicio 1.1

Implementa `chunk_fijo(texto, tamano=300, overlap=80)` sobre caracteres. Devuelve una lista de strings. Luego observa si algún chunk corta una idea.

```
# Ejercicio 1.1 — chunking fijo con solapamiento
# (a completar)
```

Código dado:

```
# El corpus fragmentado. Vive FUERA del ejercicio porque las Partes 2 y 3 lo
consumen:
# si estuviera dentro de la solución del 1.1, la versión de estudiante no lo
tendría.
chunks, origen = [], []
```

```

for nombre, texto in DOCS.items():
    for c in chunk_fijo(texto):
        chunks.append(c)
        origen.append(nombre)

for i, c in enumerate(chunks):
    print(f"[{i}] ({origen[i]}) {c[:70]}...")

```

Mira los cortes: con `overlap=80`, la idea "hasta un máximo de 5 días" transferibles aparece completa en algún chunk aunque otro la corte. Prueba `overlap=0` y vuelve a mirar.

Parte 2 — Índice (embeddings + kNN)

Código dado:

```

try:
    from sentence_transformers import SentenceTransformer
    _m = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
    def embed(ts):
        return np.asarray(_m.encode(ts, normalize_embeddings=True))
    MOTOR = "denso (sentence-transformers)"
    V = embed(chunks)
    def vec_consulta(q):
        return embed([q])[0]
except ImportError:
    from sklearn.feature_extraction.text import TfidfVectorizer
    _tf = TfidfVectorizer().fit(chunks)
    Vs = _tf.transform(chunks).toarray()
    V = Vs / (np.linalg.norm(Vs, axis=1, keepdims=True) + 1e-9)
    def vec_consulta(q):
        v = _tf.transform([q]).toarray()[0]
        return v / (np.linalg.norm(v) + 1e-9)
    MOTOR = "léxico (TF-IDF fallback)"

print("Retrieval:", MOTOR, "| índice:", V.shape)

```

El techo del modelo, medido

El chunking de la Parte 1 cuenta **caracteres** y el modelo de embeddings trunca en **tokens**: el MiniLM multilingüe procesa como mucho `max_seq_length` tokens de cada texto y descarta el resto **sin avisar**. Trescientos caracteres probablemente caben, pero «probablemente» no es una medición. Esta celda cuenta los tokens del fragmento más largo con el tokenizador del mismo modelo y los compara con su tope.

El Lab 02 del sábado hace esto por ti: fragmenta directamente en tokens de su modelo —`bge-m3`, servido en la H200 de la USFQ, con un tope de 8192— y avisa si pides un fragmento que no cabe. El MiniLM de este cuaderno es el que usa la Parte 0.b del taller para ver el truncado con tus propios ojos.

Código dado:

```

if MOTOR.startswith("denso"):
    tope = int(_m.max_seq_length)
    tokens = [len(_m.tokenizer(c)["input_ids"]) for c in chunks]
    mas_largo = int(np.argmax(tokens))
    print(f"tope del modelo: {tope} tokens")
    print(f"fragmento más largo: [{mas_largo}] con {tokens[mas_largo]} tokens "
          f"({len(chunks[mas_largo])} caracteres)")
    print("cabe entero" if tokens[mas_largo] <= tope else
          f"NO cabe: se pierden {tokens[mas_largo] - tope} tokens en silencio")
else:
    print("Sin sentence-transformers no hay modelo que truncar: el fallback TF-IDF")

```

```
lee "
    "el fragmento entero.")
```

🔗 Ejercicio 2.1

Implementa `recuperar(pregunta, k=3) → lista de (score, idx_chunk)` ordenada de mayor a menor score, usando el índice `v` y `vec_consulta(pregunta)`. La Parte 3 y el ejercicio 3.1 la llaman **con esa firma exacta**.

```
# Ejercicio 2.1 – recuperar(pregunta, k=3) → [(score, idx_chunk), ...]
# (a completar)
```

Parte 3 — Generación con contexto

El prompt del baseline: contexto delimitado + instrucción de responder SOLO con base en él + permiso explícito de decir "no sé".

Código dado:

```
# La frase de abstención es UNA en todo el curso: la misma del Lab 02
(rag_pipeline.py)
# y del golden set del sábado. Con tres redacciones distintas en tres archivos,
ningún
# detector por igualdad de cadena dispararía nunca.
ABSTENCION = "El corpus no contiene información suficiente."
```

```
PLANTILLA = """Eres un asistente de políticas internas de NimbusSoft.
Responde SOLO con base en el contexto siguiente. Si la respuesta no está en el
contexto, di exactamente: "El corpus no contiene información suficiente."
Cita el fragmento [n] que uses.
```

```
<contexto>
{contexto}
</contexto>
```

```
Pregunta: {pregunta}
Respuesta: ""
```

```
def armar_prompt(pregunta: str, k: int = 3) -> str:
    partes = [f"[{i}] {chunks[i]}" for _, i in recuperar(pregunta, k)]
    return PLANTILLA.format(contexto="\n\n".join(partes), pregunta=pregunta)
```

```
print(armar_prompt("¿puedo trabajar desde otro país?")[:900], "...")
```

Código dado:

```
PRECAPTURADAS = {
    "transferir": "Según [0], puedes transferir hasta un máximo de 5 días no
utilizados al año siguiente.",
    "exterior": "Según el contexto, sí: requiere aprobación de RRHH con 30 días de
anticipación y máximo 60 días/año.",
    "mascotas": ABSTENCION,
    "ambigua": "Según [n], los pasajes en económica premium (vuelos de más de 8
horas) requieren aprobación del gerente de área.",
}
```

```
import unicodedata
```

```
def _normalizar_texto(t: str) -> str:
    """minúsculas, sin tildes, sin puntuación: así 'informacion suficiente' e
'información suficiente.' cuentan como la misma frase."""
    t = unicodedata.normalize("NFKD", t.casefold())
```

```

t = "".join(c for c in t if not unicodedata.combining(c))
return " ".join("".join(c if c.isalnum() or c.isspace() else " " for c in
t).split())

def se_abstuvo(respuesta: str) -> bool:
    """Detecta la abstención comparando NORMALIZADO, no por igualdad de cadena."""
    return _normalizar_texto(ABSTENCION) in _normalizar_texto(respuesta)

def generar(prompt: str, etiqueta: str = "transferir") -> str:
    if os.environ.get("ANTHROPIC_API_KEY"):
        try:
            import anthropic
            r = anthropic.Anthropic().messages.create(
                model="claude-haiku-4-5", max_tokens=300,
                messages=[{"role": "user", "content": prompt}])
            return "".join(b.text for b in r.content if b.type == "text")
        except Exception as e:
            print(f"(API falló: {e})")
    # La H200 de la USFQ (vLLM, API compatible con OpenAI). Requiere GlobalProtect.
    try:
        import urllib.request
        cuerpo = json.dumps({"model": "zai-org/GLM-5.3-Flash", "temperature": 0.1,
                             # el razonamiento se paga del mismo cupo: 300 no
                             "max_tokens": 1500,
                             "messages": [{"role": "user", "content":
prompt}}]).encode()
        req = urllib.request.Request("http://172.28.230.10:12555/v1/chat/
completions",
                                     data=cuerpo, headers={"Content-Type":
"application/json"})
        with urllib.request.urlopen(req, timeout=90) as r:
            return json.loads(r.read())["choices"][0]["message"]["content"].strip()
        except Exception as e:
            print(f"(H200 no disponible: {e})")
    try:
        import urllib.request
        cuerpo = json.dumps({"model": "llama3.1:8b", "prompt": prompt, "stream":
False}).encode()
        req = urllib.request.Request("http://localhost:11434/api/generate",
                                     data=cuerpo, headers={"Content-Type":
"application/json"})
        with urllib.request.urlopen(req, timeout=90) as r:
            return json.loads(r.read())["response"]
        except Exception:
            print("(Modo inspección: sin API ni Ollama, respuesta pre-capturada)")
            return PRECAPTURADAS.get(etiqueta, f"(sin respuesta pre-capturada para
'{etiqueta}')")

def responder(pregunta: str, etiqueta: str = "transferir") -> str:
    return generar(armar_prompt(pregunta), etiqueta)

print(responder("¿Cuántos días de vacaciones puedo transferir al año siguiente?",
"transferir"))

```

🔗 Ejercicio 3.1 — Las tres pruebas de fuego

Ejecuta y analiza (usa las etiquetas "exterior" y "mascotas" en modo inspección):

1. Pregunta con respuesta clara: "¿puedo trabajar desde el exterior?"

2. Pregunta SIN respuesta en el corpus: "¿puedo llevar a mi mascota a la oficina?"
3. Pregunta ambigua entre documentos: "¿qué necesita aprobación del gerente?"

Para cada una anota: ¿qué chunks recuperó? ¿la respuesta se apoya en ellos?

Código dado:

```
# Ejercicio 3.1 – ejecución de las tres pruebas

PRUEBAS = [
    ("¿puedo trabajar desde el exterior?", "exterior"),          # respuesta clara
    ("¿puedo llevar a mi mascota a la oficina?", "mascotas"),    # fuera del corpus
    ("¿qué necesita aprobación del gerente?", "ambigua"),        # ambigua entre
    documentos
]

for pregunta, etiqueta in PRUEBAS:
    print(f"\n➤ {pregunta}")
    for score, i in recuperar(pregunta):
        print(f"    {score:.3f}  [{i}] ({origen[i]}) {chunks[i][:70]}...")
    respuesta = responder(pregunta, etiqueta)
    print(f"    respuesta : {respuesta}")
    print(f"    ¿se abstuvo? {se_abstuvo(respuesta)}")
```

Punto clave de la prueba 2: el mejor comportamiento posible es la frase de abstención, `ABSTENCION` («El corpus no contiene información suficiente»), la misma que evalúa el Lab 02. Un RAG que siempre responde algo es un generador de alucinaciones con citas decorativas.

```
# Diagnóstico final (comentario):
# De las 4 cajas del pipeline (ingesta / chunking / retrieval / generación),
# ¿cuál te parece más frágil en TU proyecto y qué harías para vigilarla?
#
```

Cierre

Tienes un RAG completo: chunking con overlap, índice, retrieval top-k y generación con contexto delimitado y derecho a decir "no sé".

Mañana: ¿este RAG es *bueno*? Métricas de retrieval (Hit Rate, MRR, Recall@k), golden sets, y las extensiones que compiten contra tu baseline.